



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

CAPSTONE PROJECT REPORT

LOGISTICS AND SUPPLY CHAIN CARBON FOOTPRINT TRACKING SYSTEM

A C++17 Object-Oriented Offline System for Pre-Transit Carbon-Aware Logistics Optimization

Submitted in partial fulfilment of the requirements for the course

C++ Programming

towards the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE

Submitted By	Register Number
A. Virender Singh	1911260054
J. Raksakanth	1911260099
T. Tarun	1924260001

Under the Supervision of
Dr. PARIMALA KANAGA DEVAN K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, A. Virender Singh, J. Raksakanth and T. Tarun, students of SIMATS Engineering, hereby declare that the project report entitled “Logistics and Supply Chain Carbon Footprint Tracking System” is prepared as part of our academic capstone work in C++ Programming.

The report documents the problem context, objectives, requirements, system design, object-oriented architecture, route-optimization approach, persistence strategy, prototype workflow, testing plan, project planning and learning outcomes associated with the proposed logistics carbon-footprint tracking system.

Where the supplied project material describes a feature as proposed, planned, prototype-level or future work, this report preserves that distinction. Reported performance figures are identified as project-reported outcomes unless supported by a reproducible benchmark record.

Name	Register Number	Signature
A. Virender Singh	1911260054	_____
J. Raksakanth	1911260099	_____
T. Tarun	1924260001	_____

Place: SIMATS Engineering

Date: _____



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Logistics and Supply Chain Carbon Footprint Tracking System” has been carried out by A. Virender Singh, J. Raksakanth and T. Tarun under the guidance and supervision of Dr. PARIMALA KANAGA DEVAN K, and is submitted in partial fulfilment of the academic requirements of SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences.

The work addresses the application of C++17 object-oriented programming concepts to a logistics decision-support problem involving shipment management, transportation-mode representation, carbon estimation and route optimization.

Project Guide / Supervisor	Designation / Area
Dr. PARIMALA KANAGA DEVAN K	Project Supervisor / C++ Programming

Internal Examiner: _____

External Examiner: _____

Viva-Voce Date: _____

Signature of Guide

ACKNOWLEDGEMENT

We express our sincere gratitude to the management and faculty of SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, for providing the academic environment, facilities and encouragement necessary for completing this capstone project.

We are especially thankful to our project supervisor, Dr. PARIMALA KANAGA DEVAN K, for valuable guidance, constructive feedback and continuous encouragement during the planning and development of the work. The suggestions received during the design stage helped us understand how object-oriented programming principles could be mapped to a practical logistics and sustainability problem.

We also acknowledge the support of the department faculty, project coordinators, laboratory staff, classmates, parents and friends who contributed through discussions, reviews, testing suggestions and motivation.

This project provided an opportunity to connect software engineering fundamentals with graph algorithms, data persistence, authentication, environmental accounting concepts and logistics decision-making. The process improved our ability to decompose a complex problem into manageable modules and communicate a technical solution in a structured academic format.

Student	Signature
A. Virender Singh	_____
J. Raksakanth	_____
T. Tarun	_____

ABSTRACT

Freight logistics across road, rail, sea and air contributes to transportation-related carbon emissions. Conventional logistics decisions may prioritize cost, delivery time or operational convenience, while environmental impact is often calculated after transportation has occurred. The Logistics and Supply Chain Carbon Footprint Tracking System is proposed as a lightweight C++17 offline application that introduces carbon visibility before shipment dispatch.

The system combines shipment management, transport-mode representation, emission-factor processing and graph-based route optimization. Its central decision model uses a dual-weighted cost function, $C = \alpha \times \text{Time} + \beta \times \text{CO}_2$, allowing delivery time and environmental impact to be considered simultaneously. Dijkstra's algorithm is used to identify a route with minimum combined cost over a weighted logistics graph.

The architecture is organized into three broad functional areas: the Carbon Engine and Green Path Optimizer; the Logistics and Shipment Operations Engine; and the Authentication, UI and Exception Engine. The design identifies Graph, PathOptimizer, TransportMode, Shipment, DataLoader, PersistenceManager, AuthManager and ExceptionController as major classes. RoadTransport, RailTransport, SeaTransport and AirTransport are specialized transport types.

Binary and CSV persistence are included to support offline operation and human-readable data exchange. The project material reports sub-millisecond computation and approximately 40%–73% carbon-output reduction in long-haul/intermodal scenarios. Because the supplied material does not include a reproducible benchmark protocol, these figures are presented in this report as project-reported outcomes rather than independently verified measurements.

Future extensions include live telemetry, IoT integration, traffic and weather data, machine-learning prediction, cloud synchronization and richer visualization. The project demonstrates how OOP concepts can be combined with graph algorithms and environmental decision criteria to create a modular logistics software concept.

Keywords: C++17; OOP; Carbon Footprint; Logistics; Route Optimization; Dijkstra; Intermodal Transport; Offline System

TABLE OF CONTENTS

1	INTRODUCTION	7
1.1	Background Information	7
1.2	Project Objectives	7
1.3	Significance of the Project	8
1.4	Scope of the Project	8
1.5	Methodology Overview	8
1.6	Report Organization	9
2	PROBLEM IDENTIFICATION AND ANALYSIS	10
2.1	Problem Statement	10
2.2	Evidence and Context	10
2.3	Stakeholders	11
2.4	Functional Requirements	11
2.5	Non-Functional Requirements	12
2.6	Research Gap	12
3	SOLUTION DESIGN AND IMPLEMENTATION	13
3.1	Design Approach	13
3.2	Tools and Technologies	13
3.3	System Architecture	14
3.4	Module Design	15
3.5	OOP Design	16
3.6	UML / Class Relationships	17
3.7	Carbon Calculation	18
3.8	Route Optimization	19
3.9	Persistence and Security	20
4	IMPLEMENTATION, TESTING AND RESULTS	21
4.1	Implementation Plan	21
4.2	Prototype Workflow	22
4.3	Test Strategy	23
4.4	Test Cases	24
4.5	Results and Discussion	25
4.6	Limitations	25
5	RECOMMENDATIONS AND FUTURE SCOPE	26
5.1	Recommendations	26
5.2	Future Enhancements	26
6	REFLECTION ON LEARNING AND DEVELOPMENT	27
6.1	Technical Learning	27
6.2	Challenges and Solutions	28
6.3	Engineering Standards	28
6.4	Industry Insights	29
7	PROJECT PLANNING AND RUBRIC ALIGNMENT	30
7.1	Timeline	30
7.2	Team Responsibilities	30
7.3	Rubric Alignment	31
8	CONCLUSION AND REFERENCES	32
	Appendix A OOP Class Details	33
	Appendix B Prototype / Console Flow	3

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND INFORMATION

Transportation is an essential component of supply-chain activity. Freight may move through road, rail, sea and air networks, often involving transfers between modes. Each alternative can differ in delivery time, distance, operational characteristics and carbon intensity. A logistics planner therefore faces a multi-objective decision: a route that is fastest may not be the route with the lowest environmental impact.

The project responds to this decision problem by placing carbon information inside the route-selection process. Instead of treating carbon as a report generated after delivery, the proposed system estimates the transportation-related footprint before dispatch. This supports a planning workflow in which a user can compare alternatives and select a route based on a combined time-and-carbon objective.

1.2 PROJECT OBJECTIVES

- Develop a lightweight offline logistics application using C++17.
- Represent shipments and transport modes using object-oriented design.
- Estimate transportation-related CO₂ using emission factors and shipment information.
- Represent logistics routes as a weighted graph.
- Optimize route selection using a combined time and carbon objective.
- Support road, rail, sea and air transport alternatives.
- Provide shipment create, view, update and delete operations.
- Provide binary persistence and CSV import/export.
- Provide authentication, role-based access, validation and exception handling.

1.3 SIGNIFICANCE OF THE PROJECT

The significance of the project is its attempt to combine operational and environmental considerations within one decision-support workflow. The system is intentionally lightweight and offline, making the design suitable for academic demonstration and environments where permanent cloud connectivity is not assumed.

1.4 SCOPE OF THE PROJECT

In Scope	Description	Status
Shipment management	Create, view, update and delete shipment records.	Core design
Transport modes	Road, rail, sea and air abstractions.	Core design
Carbon estimation	Use mode-specific emission factors and shipment data.	Core design
Route optimization	Weighted graph and dual-objective route cost.	Core design
Intermodal evaluation	Consider transfer hubs and multi-mode alternatives.	Planned/design feature
Persistence	Binary records and CSV export/import.	Core design
Authentication	Credentials, role checks and authorization.	Core design
Exception handling	Input validation, sanitization and error logging.	Core design
Live telemetry	Real-time sensors and streaming data.	Future scope
Cloud synchronization	Remote storage and live synchronization.	Future scope

1.5 METHODOLOGY OVERVIEW

The methodology begins with problem identification and literature comparison. Requirements are then translated into functional modules. The software architecture separates business logic from data storage and user interaction. OOP classes are designed around focused responsibilities. A weighted graph is used for route representation, while a dual-objective cost function supplies the edge weight used by the optimizer.

1. Identify the carbon-visibility gap in pre-dispatch logistics decisions.
2. Define users, requirements, constraints and project scope.
3. Model shipments, routes, modes and persistence entities.
4. Design the Carbon Engine and Path Optimizer.
5. Define authentication, UI flow and exception handling.
6. Define binary and CSV data storage.
7. Prepare test cases for CRUD, calculations, routing and validation.
8. Document results, limitations and future improvements.

1.6 REPORT ORGANIZATION

Chapter 1 introduces the project and its scope. Chapter 2 analyzes the problem, stakeholders and requirements. Chapter 3 presents the architecture, OOP design, carbon model and route optimization. Chapter 4 explains implementation planning, prototype flow, testing and results. Chapter 5 presents recommendations and future scope. Chapter 6 records learning and professional development. Chapter 7 covers planning and rubric alignment. Chapter 8 concludes the report and lists references, followed by appendices.

CHAPTER 2: PROBLEM IDENTIFICATION AND ANALYSIS

2.1 PROBLEM STATEMENT

The primary problem addressed by the project is limited visibility of carbon impact at the time a freight route is selected. A planner may know the destination, shipment weight and expected delivery requirement but still lack an integrated mechanism for comparing the environmental impact of alternative transport modes and routes before dispatch.

The proposed system transforms the logistics network into a graph in which routes have associated time and carbon characteristics. These characteristics are combined into a single decision cost, allowing an optimizer to select a route that balances delivery performance and environmental impact.

2.2 EVIDENCE AND CONTEXT

The project material identifies road, rail, sea and air transportation as relevant contributors to transportation-related CO₂. It also identifies a research gap between post-transit carbon accounting and pre-dispatch route optimization. Literature reviewed for the project covers telemetry/cloud approaches, payload-weight regression, multi-objective logistics optimization and ISO-oriented accounting.

2.3 STAKEHOLDERS

Stakeholder	Primary Need	Expected Interaction
Logistics planner	Compare route/time/carbon alternatives.	Route optimization and reports
Shipment operator	Maintain accurate shipment records.	CRUD operations
Administrator	Control access and roles.	Authentication/user management
Auditor/reviewer	Inspect data and assumptions.	CSV reports and records
Organization	Reduce environmental impact without ignoring delivery needs.	Decision support
Developer/maintainer	Extend modules safely.	OOP architecture and persistence

2.4 FUNCTIONAL REQUIREMENTS

ID	Requirement	Description
FR01	Login	Authenticate a user before protected functions are accessed.
FR02	Role authorization	Permit actions according to user role.
FR03	Create shipment	Store shipment ID, source, destination, weight and related information.
FR04	View shipment	Display stored shipment details.
FR05	Update shipment	Modify an existing shipment record.
FR06	Delete shipment	Remove a selected shipment record.
FR07	Load routes	Read route/network information.
FR08	Load emission factors	Read mode-specific carbon parameters.
FR09	Calculate carbon	Estimate emissions for candidate transport segments.
FR10	Optimize route	Calculate combined route cost and select a route.
FR11	Intermodal evaluation	Evaluate transfer hubs and mode changes.
FR12	Persistence	Save and load shipment data.
FR13	CSV export/import	Support human-readable data exchange.
FR14	Error handling	Validate inputs and handle file/runtime errors.

2.5 NON-FUNCTIONAL REQUIREMENTS

Category	Requirement
Performance	Route calculations should be efficient for the intended academic dataset.
Usability	Menu-driven workflow should be understandable to a first-time operator.
Reliability	Invalid inputs and missing files should be handled safely.
Maintainability	Modules should have focused responsibilities.
Portability	C++17-compatible environment should be sufficient.
Offline operation	Core functions should not require continuous internet access.
Data integrity	Persistence should preserve required shipment fields.
Security	Authentication data and role checks should restrict protected operations.

2.6 RESEARCH GAP

The literature comparison used in the project presentation identifies several limitations in existing approaches. Telemetry-oriented systems may depend on cloud infrastructure. Payload regression can focus on road transport rather than a complete multi-modal network. Multi-objective optimization may emphasize time and cost without making carbon a primary objective. Conventional accounting approaches may concentrate on post-transit measurement.

Reference / Study	Method	Strength	Limitation Identified	Project Response
Gupta & Kumar (2026)	Telemetry + cloud	Live visibility	Cloud dependency	Offline persistence
Al-Mansoor et al. (2025)	Payload regression	Payload effects	Road-only scope	Multi-modal model
Fernandez & Smith (2025)	Multi-objective optimization	Trade-off analysis	Carbon not primary	Time + CO ₂ cost
Zhang & Liu (2024)	ISO-oriented accounting	Reporting framework	Post-transit focus	Pre-dispatch decision support

2.7 PROBLEM ANALYSIS SUMMARY

- Carbon should be visible before dispatch rather than only after delivery.
- Transport mode can materially change the carbon characteristics of a route.
- Time and environmental objectives can conflict and require an explicit trade-off.
- An offline academic system can demonstrate the decision process without mandatory cloud services.
- Modular OOP design is appropriate because shipment, route, mode, persistence and authentication concerns are distinct.

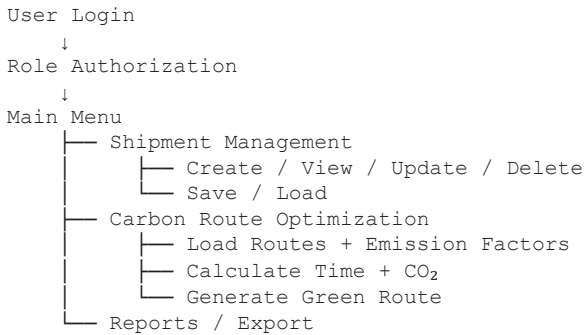
The resulting design is therefore not intended to replace enterprise logistics platforms. It is an academic decision-support prototype that demonstrates how carbon-aware route selection can be structured in C++17.

CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION

3.1 DESIGN APPROACH

The solution is designed as an offline, menu-driven C++17 application. The design emphasizes separation of responsibilities. Carbon computation and route optimization are kept separate from shipment CRUD. Authentication and exception handling are also isolated so that changes in one concern do not require rewriting unrelated modules.

The overall workflow can be represented as:

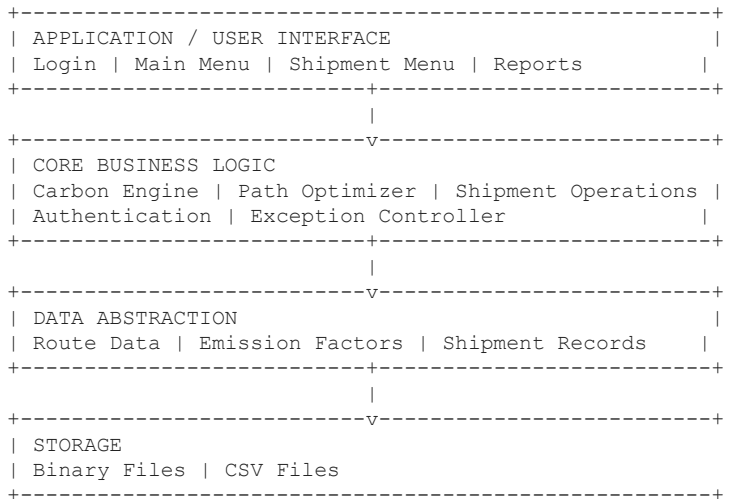


3.2 TOOLS AND TECHNOLOGIES

Technology / Concept	Role in the Project
C++17	Primary programming language.
Classes / Objects	Core OOP structure.
Inheritance	Specialized transport modes.
Polymorphism	Common transport interface with mode-specific behavior.
Graph / adjacency list	Logistics network representation.
Dijkstra algorithm	Minimum combined-cost route selection.
std::fstream	File input/output.
Binary files	Persistent application records.
CSV	Human-readable import/export.

3.3 SYSTEM ARCHITECTURE

The architecture can be understood as four cooperating layers. The application layer presents the menu and report workflow. The core business layer performs carbon calculations, shipment operations, route optimization and access control. The data abstraction layer manages routes, emission factors and shipment records. The storage layer provides binary and CSV files.



3.4 ARCHITECTURAL PRINCIPLES

- Single responsibility: each major class has a focused purpose.
- Abstraction: common transport behavior is represented by TransportMode.
- Encapsulation: shipment state is managed through controlled operations.
- Extensibility: new transport modes can be added without redesigning the entire optimizer.
- Persistence separation: storage details are isolated from business logic.
- Validation boundary: user input is checked before it reaches core calculations.

3.5 MODULE DESIGN

Module	Main Responsibility	Inputs	Outputs
Authentication	Verify identity and role.	Username, password	Authenticated role / error
Shipment Operations	CRUD on shipment records.	Shipment fields	Stored/updated shipment
Data Loader	Read routes and emission factors.	Binary/CSV files	In-memory data
Carbon Engine	Estimate emissions.	Distance, weight, mode factor	CO ₂ estimate
Path Optimizer	Select minimum combined-cost path.	Graph, time, CO ₂ , α , β	Selected route + cost
Persistence	Save/load records and export/import.	Application data	Files / records
Exception Controller	Validate and handle errors.	User/file input	Safe response/log
Reports	Present route and carbon results.	Optimization output	Readable report

3.6 DATA FLOW

A typical route-optimization request begins when an authenticated user selects a shipment and requests a green-route analysis. Shipment information is read from the application state. Route and emission data are loaded through DataLoader. Candidate edges are assigned a combined cost based on time and CO₂. PathOptimizer then evaluates the graph and returns a selected route. The report layer presents the route, time, estimated CO₂ and combined cost.

```
Shipment → Route Request
      ↓
DataLoader → Routes + Emission Factors
      ↓
Carbon Engine → Segment CO2
      ↓
Path Optimizer →  $C = \alpha T + \beta \text{CO}_2$ 
      ↓
Green Route Report → Route / Time / CO2 / Cost
```

3.7 OBJECT-ORIENTED DESIGN

The system uses OOP because the domain naturally contains entities with state and behavior. A Shipment has identifying and routing information and therefore benefits from encapsulated operations. A transport mode has common behavior but different emission characteristics, making inheritance and polymorphism appropriate. Graph and PathOptimizer represent algorithmic concerns and can be developed independently of user-interface code.

Class	Key Attributes	Representative Methods
Graph	adjacencyList, nodes, edges	addNode(), addEdge(), isReachable(), getNeighbors()
PathOptimizer	graph, alpha, beta	calculateCost(), optimizeRoute(), evaluateIntermodalHub(), generateGreenRoute()
TransportMode	modeName, emissionFactor	calculateEmission(), getMode()
RoadTransport	mode-specific data	calculateEmission()
RailTransport	mode-specific data	calculateEmission()
SeaTransport	mode-specific data	calculateEmission()
AirTransport	mode-specific data	calculateEmission()
Shipment	shipmentId, source, destination, goodsWeightKg	createShipment(), viewShipment(), updateShipment(), deleteShipment()
DataLoader	routeFile, emissionFile	loadRoutes(), loadEmissionFactors(), loadShipmentData()
PersistenceManager	binaryFile, csvFile	saveBinary(), loadBinary(), exportCSV(), importCSV()
AuthManager	username, role, credentials	authenticate(), authorize(), loadCredentials()
ExceptionHandler	validation state	validateInput(), sanitizeInput(), handleException(), logError()

The class list and method names are based on the project presentation's documented OOP design. Actual implementation should be cross-checked against the final C++ source.

3.8 APPLICATION OF OOP CONCEPTS

OOP Concept	Application in the System	Benefit
Encapsulation	Shipment data and operations are kept within Shipment.	Reduces unintended state changes.
Abstraction	TransportMode represents common transport behavior.	Hides mode-specific details behind an interface.
Inheritance	RoadTransport, RailTransport, SeaTransport and AirTransport specialize TransportMode.	Avoids duplicated common structure.
Polymorphism	A common emission calculation can be invoked through TransportMode.	Allows mode-specific behavior at runtime.
Modularity	Carbon, shipment, authentication and persistence concerns are separated.	Improves maintainability and testing.

3.9 CLASS RELATIONSHIP DESCRIPTION

```
TransportMode
├── RoadTransport
├── RailTransport
├── SeaTransport
└── AirTransport

Shipment ---- uses ----> PathOptimizer
                        |
                        +---- uses ----> Graph
                        |
                        +---- uses ----> TransportMode

DataLoader ---- supplies ----> Graph / Carbon Engine
PersistenceManager ---- stores ----> Shipment data
AuthManager ---- protects ----> Main Menu functions
ExceptionHandler ---- validates ----> User/File input
```

The relationship structure is designed to minimize coupling. PathOptimizer depends on abstract transport behavior rather than requiring separate routing code for every mode. PersistenceManager is also isolated so that changing from binary files to another storage mechanism would not require rewriting the core routing logic.

3.10 UML / CLASS-DIAGRAM SPECIFICATION

The following textual UML specification can be used to recreate a formal class diagram in a drawing tool. It is included because the report must communicate relationships even when a graphical UML image is not embedded.

```
CLASS Graph
- adjacencyList
- nodes
- edges
+ addNode()
+ addEdge()
+ isReachable()
+ getNeighbors()

CLASS PathOptimizer
- graph
- alpha
- beta
+ calculateCost()
+ optimizeRoute()
+ evaluateIntermodalHub()
+ generateGreenRoute()

ABSTRACT CLASS TransportMode
- modeName
- emissionFactor
+ calculateEmission()
+ getMode()

Shipment
- shipmentId
- source
- destination
- goodsWeightKg
+ createShipment()
+ viewShipment()
+ updateShipment()
+ deleteShipment()
```

3.11 DESIGN RATIONALE

Graph and PathOptimizer are separated because representing a network and selecting a path are different responsibilities. TransportMode is abstract because the optimizer should work with a common interface. Shipment is responsible for its own business record operations. PersistenceManager is separate because file-format decisions should not leak into domain classes.

3.12 CARBON CALCULATION MODEL

The carbon engine estimates emissions using transport-mode information and shipment characteristics. The exact emission-factor dataset is not reproduced here because the supplied project material describes the factor inputs conceptually rather than providing a validated numerical dataset.

A generic segment-level calculation can be represented as:

$$\text{Segment CO}_2 = \text{Distance} \times \text{Shipment Activity} \times \text{Mode Emission Factor}$$

where the activity term may incorporate shipment weight or another defined unit consistent with the selected emission-factor dataset.

The route optimizer then combines environmental impact with delivery time:

$$C = \alpha \times \text{Time} + \beta \times \text{CO}_2$$

α = relative importance assigned to delivery time

β = relative importance assigned to carbon impact

3.13 INTERPRETING α AND β

Setting	Meaning	Expected Effect
Higher α	Time is more important.	Faster routes are favored.
Higher β	Carbon is more important.	Lower-carbon routes are favored.
Balanced α , β	Both objectives matter.	Trade-off route may be selected.
$\beta = 0$	Carbon ignored.	Optimization behaves as time-focused.
$\alpha = 0$	Time ignored.	Optimization behaves as carbon-focused.

For a scientifically reproducible experiment, α , β , units and emission-factor definitions must be stated explicitly and kept consistent across all test cases.

3.14 ROUTE OPTIMIZATION

The logistics network is represented as a weighted graph. Nodes may represent locations or hubs, while edges represent transportation segments. Each edge can carry time and carbon information. The combined cost of an edge is used as the weight for route optimization.

Dijkstra-Based Procedure

9. Initialize the source node with distance zero and all other nodes with infinity.
10. Select the unvisited node with the smallest current combined cost.
11. Examine each neighboring edge.
12. Calculate the candidate cost using time and CO₂.
13. Relax the edge if the candidate cost is smaller than the stored cost.
14. Continue until the destination is finalized or all reachable nodes are processed.
15. Reconstruct the route from predecessor information.

```
edgeCost = alpha * edgeTime + beta * edgeCO2
candidate = currentCost + edgeCost
if candidate < bestCost[neighbor]:
    bestCost[neighbor] = candidate
    predecessor[neighbor] = current
```

INTERMODAL ROUTING

Intermodal routing extends the same idea to routes where the transport mode changes at a hub. A transfer may improve carbon performance but add time or handling overhead. The optimizer therefore needs to represent the transfer consistently rather than treating every route as a single-mode path.

3.15 DATA PERSISTENCE

The project design uses binary files for persistent records and CSV files for human-readable inspection or exchange. Binary storage is useful for compact structured records, while CSV is useful when users need to review or export data using ordinary spreadsheet or text tools.

Operation	Binary Persistence	CSV Persistence
Save	Write structured records.	Write tabular fields.
Load	Restore application records.	Read rows and reconstruct records.
Export	Not intended for manual inspection.	Human-readable output.
Import	Can restore structured data.	Useful for external tabular input.

3.16 AUTHENTICATION AND AUTHORIZATION

AuthManager is responsible for verifying credentials and determining the role associated with an authenticated user. Protected functions can then be restricted according to role. For a production implementation, passwords should not be stored as plain text; a secure password-hashing mechanism and appropriate credential storage would be required.

3.17 EXCEPTION HANDLING

- Reject empty or malformed required fields.
- Validate numeric ranges such as shipment weight.
- Handle missing route/emission files.
- Handle invalid menu selections.
- Prevent operations on non-existent shipment IDs.
- Report file and runtime errors without abruptly terminating the workflow.

CHAPTER 4: IMPLEMENTATION, TESTING AND RESULTS

4.1 IMPLEMENTATION PLAN

Implementation should proceed incrementally. The first stage establishes the data model and transport abstraction. The second stage builds shipment CRUD and persistence. The third stage integrates the graph and carbon engine. The fourth stage adds route optimization. The fifth stage integrates authentication, validation and reporting. Integration testing is performed after each major module is connected.

Stage	Main Deliverable	Verification
1	OOP skeleton	Compile classes and basic object creation.
2	Shipment CRUD	Create/view/update/delete tests.
3	Persistence	Save/load and CSV tests.
4	Carbon engine	Known-input calculation tests.
5	Graph + optimizer	Route correctness tests.
6	Authentication	Valid/invalid credential tests.
7	Integration	End-to-end menu workflow.
8	Documentation	Report, presentation and viva material.

4.2 IMPLEMENTATION CHECKLIST

- Compile under a C++17-compatible environment.
- Confirm all required headers and source files are organized.
- Verify that each class has a single clear responsibility.
- Validate input before calculation.
- Test persistence after application restart.
- Verify route reconstruction and output formatting.
- Document assumptions used by emission factors.

4.3 PROTOTYPE WORKFLOW

The documented prototype is a menu-driven console-style workflow. The source presentation describes the interface flow as a prototype/proposed flow rather than evidence of a completed graphical interface.

LOGIN

```
LOGISTICS CARBON SYSTEM
Username: _____
Password: _____
[1] Login
[2] Exit
```

MAIN MENU

```
MAIN MENU
[1] Shipment Management
[2] Carbon Route Optimization
[3] Reports
[4] User Management
[5] Exit
```

SHIPMENT MANAGEMENT

```
SHIPMENT MANAGEMENT
[1] Add Shipment
[2] View Shipments
[3] Update Shipment
[4] Delete Shipment
```

GREEN ROUTE REPORT

```
GREEN ROUTE REPORT
Route: <selected route>
Time: <calculated time>
CO2: <calculated carbon>
Cost:  $\alpha \cdot T + \beta \cdot CO_2$ 
Recommendation: <selected route>
```

4.4 TEST STRATEGY

Testing should verify each module independently before end-to-end integration. Unit-style checks are appropriate for carbon calculations and class methods. Integration tests should confirm that shipment data can move through persistence, route analysis and reporting. Negative tests are particularly important for authentication, invalid input and missing files.

Test Level	Purpose	Examples
Unit	Verify individual methods.	Shipment update; emission calculation.
Integration	Verify modules work together.	Shipment → optimizer → report.
System	Verify complete user workflow.	Login → CRUD → optimize → export.
Negative	Verify safe failure.	Invalid login; bad weight; missing file.
Regression	Ensure changes do not break existing functions.	Repeat core suite after modifications.

4.5 TESTING PRINCIPLES

- Use known input/output pairs for calculation tests.
- Test boundary values such as zero, minimum and maximum accepted weight.
- Test unreachable destinations.
- Test multiple routes with equal cost.
- Test invalid shipment IDs.
- Test persistence across save/load cycles.
- Test role restrictions.
- Test malformed or missing data files.

4.6 TEST CASES

TC ID	Scenario	Input	Expected Result
TC01	Valid login	Correct credentials	User authenticated.
TC02	Invalid login	Incorrect credentials	Access denied.
TC03	Create shipment	Valid ID/source/destination/weight	Shipment stored.
TC04	Update shipment	Existing ID + changed field	Record updated.
TC05	Delete shipment	Existing ID	Record removed.
TC06	Invalid weight	Negative/non-numeric	Validation error.
TC07	Save/load	Valid shipment set	Loaded data matches saved data.
TC08	CSV export	Shipment set	Readable CSV generated.
TC09	Route optimization	Connected graph	Minimum combined-cost route returned.
TC10	Unreachable route	Disconnected destination	No-route message.
TC11	Carbon calculation	Known distance/factor	Expected calculation.
TC12	Intermodal route	Hub + mode change	Route evaluated with transfer effect.
TC13	Missing file	Non-existent data file	Handled error.
TC14	Unauthorized action	Restricted role	Operation rejected.
TC15	Invalid menu	Out-of-range choice	Input rejected/re-prompted.

The table is a test specification for the documented system design. It should be marked Pass/Fail with actual execution evidence after the final C++ build is available.

4.7 RESULTS AND DISCUSSION

The supplied project presentation reports two headline outcomes: sub-millisecond computation and approximately 40%–73% carbon-output reduction in long-haul/intermodal scenarios. These figures are retained as project-reported outcomes, but the available project material does not contain a complete reproducible benchmark protocol, dataset, hardware specification or test log. Therefore, this report avoids presenting them as independently verified experimental results.

Reported Outcome	How It Should Be Interpreted
Sub-millisecond computation	Project-reported performance claim; requires reproducible benchmark evidence.
40%–73% carbon-output reduction	Project-reported/target range; requires baseline and route-comparison evidence.
Pre-dispatch carbon visibility	Core design objective and primary contribution.
Time + CO ₂ optimization	Central mathematical model.
Multi-modal transport support	Core architectural design.

4.8 LIMITATIONS

- Emission factors are only as reliable as their source, units and update frequency.
- The α and β weights require calibration for different operational priorities.
- Static route data cannot represent live traffic, weather or unexpected disruptions.
- Offline storage limits synchronization across multiple users or locations.
- Reported carbon-reduction and performance figures require reproducible validation.
- The prototype workflow is not equivalent to a production enterprise logistics platform.

CHAPTER 5: RECOMMENDATIONS AND FUTURE SCOPE

5.1 RECOMMENDATIONS

- Define and document the exact units and sources of all emission factors.
- Create a reproducible benchmark dataset for route and carbon comparisons.
- Record α and β for every experiment so results can be replicated.
- Add automated unit and integration tests to the C++ project.
- Validate binary and CSV persistence with repeated save/load cycles.
- Use secure credential handling rather than plain-text passwords.
- Keep the core routing engine independent from the user interface.

5.2 FUTURE ENHANCEMENTS

Enhancement	Potential Value	Priority
IoT telemetry	Near-real-time shipment and vehicle information.	High
Traffic integration	More realistic travel-time estimates.	High
Weather integration	Account for weather-related route effects.	Medium
Cloud synchronization	Shared records across locations.	Medium
Machine learning	Predict time, demand or emission patterns.	Medium
Dashboard visualization	Improve decision interpretation.	High
Dynamic emission factors	Keep calculations current.	High
Advanced intermodal costs	Model handling/transfer effects.	Medium

5.3 VALIDATION ROADMAP

16. Freeze the input dataset and emission-factor version.
17. Define baseline routes and optimized routes.
18. Specify α and β before running comparisons.
19. Measure execution time over repeated runs.
20. Calculate carbon difference using a documented baseline.
21. Store test results and configuration.
22. Repeat tests on multiple network sizes.
23. Report mean, minimum and maximum values rather than a single run.

CHAPTER 6: REFLECTION ON LEARNING AND DEVELOPMENT

6.1 TECHNICAL LEARNING

The project provided practical experience in converting a real-world problem into software abstractions. Instead of beginning with code, the team first identified the domain entities and operations: shipments, routes, modes, emissions, users and persistence. This approach reinforced the importance of requirements and architecture before implementation.

Graph algorithms became relevant because logistics can be naturally represented as a network. The dual-objective cost function demonstrated that an algorithm can optimize a business rule defined outside the algorithm itself. The project also strengthened understanding of inheritance and polymorphism through transport-mode specialization.

6.2 SOFTWARE ENGINEERING LEARNING

The project highlighted the importance of separation of concerns. Authentication should not be mixed with route calculation, and file formats should not dictate the internal representation of shipments. This led to the proposed separation between AuthManager, PersistenceManager, Shipment, Carbon Engine and PathOptimizer.

6.3 COMMUNICATION LEARNING

Preparing the report requires technical claims to be supported by appropriate evidence. A major documentation lesson is to distinguish an implemented feature, a prototype feature, a design proposal and a future enhancement. This distinction improves credibility and makes the project easier for another developer or evaluator to understand.

6.4 CHALLENGES AND SOLUTIONS

Challenge	Reason	Approach / Learning
Choosing α and β	Time and carbon have different meanings and scales.	Document units and perform sensitivity analysis.
Multi-modal modeling	Modes have different characteristics.	Use TransportMode abstraction and specialization.
Persistence	Records must survive program sessions.	Separate PersistenceManager from domain logic.
Input validation	User input can break assumptions.	Validate at the application boundary.
Route optimization	Many possible paths can exist.	Represent the network as a weighted graph.
Carbon verification	Factors vary by source and unit.	Document factor source, unit and version.
Performance claims	Single timing results are unreliable.	Use repeated reproducible benchmarks.

6.5 APPLICATION OF ENGINEERING STANDARDS

The project presentation references ISO 14064-1:2018 and ISO 14040:2006 in connection with greenhouse-gas accounting and life-cycle principles. These references provide context for disciplined environmental measurement, while the software engineering side benefits from modular design, validation, testing and documentation.

6.6 PROFESSIONAL RESPONSIBILITY

Environmental software can influence operational decisions, so numerical results should not be presented with greater certainty than the underlying data supports. The project therefore treats emission factors, assumptions, optimization weights and benchmark evidence as explicit documentation requirements.

6.7 INSIGHTS INTO INDUSTRY

The project illustrates an important industry tension: logistics systems are expected to deliver efficiently while organizations increasingly need to account for environmental impact. A decision-support tool must therefore provide information that is operationally useful rather than merely producing a sustainability report.

The architecture also reflects industry software concerns. Data needs to persist reliably, users need controlled access, invalid input needs to be handled safely, and the system should be maintainable as requirements change. The OOP design supports these goals by isolating responsibilities.

6.8 PERSONAL DEVELOPMENT SUMMARY

- Improved ability to decompose a real-world problem.
- Improved understanding of C++ classes and relationships.
- Improved understanding of inheritance and polymorphism.
- Improved ability to connect graph algorithms with domain objectives.
- Improved awareness of data persistence and validation.
- Improved technical documentation and presentation skills.
- Improved awareness of the difference between design claims and verified experimental results.

6.9 CONCLUSION OF PERSONAL DEVELOPMENT

The capstone experience demonstrates that a successful software project requires more than implementation. Requirements, architecture, data assumptions, testing, documentation and communication are all necessary. The project provided a practical context in which these skills could be developed together.

CHAPTER 7: PROJECT PLANNING AND RUBRIC ALIGNMENT

7.1 PROJECT TIMELINE

Period	Milestone	Activities
Week 1	Requirements + Literature	Problem definition, scope, objectives and literature comparison.
Week 2	OOP + UML	Class responsibilities, inheritance and relationships.
Weeks 3–4	Carbon Engine + Graph	Emission logic, graph representation and route cost.
Weeks 4–5	Shipment CRUD + Persistence	Record operations, binary and CSV handling.
Week 6	Authentication + Exceptions	Login, roles, validation and error handling.
Week 7	Integration + Testing	Connect modules and execute test cases.
Week 8	Documentation + Presentation	Report, presentation, demonstration and viva preparation.

7.2 PROPOSED TEAM RESPONSIBILITIES

Member	Primary Responsibility	Shared Responsibilities
A. Virender Singh	Carbon engine, graph and path optimization.	Integration, testing, presentation.
J. Raksakanth	Shipment CRUD, data structures, persistence.	Integration, testing, documentation.
T. Tarun	Authentication, UI flow, validation, exceptions.	Integration, testing, presentation.

The responsibilities above are presented as the proposed allocation documented in the project material. Replace them with actual work records if required by the department.

7.3 RUBRIC ALIGNMENT

Criterion	Evidence in This Report	Marks
Problem Identification	Chapters 1 and 2: context, gap, problem statement.	10
Scope & Requirements	Scope, functional and non-functional requirements.	10
OOP Design	Classes, attributes, methods and responsibilities.	20
UML / Class Diagram	Textual UML specification and relationship description.	15
Application of OOP Concepts	Encapsulation, abstraction, inheritance, polymorphism, modularity.	15
Project Planning	Timeline and team responsibility plan.	10
Prototype / Demonstration	Prototype workflow and reporting flow.	10
Documentation & Presentation	Full report structure, references and viva preparation.	10
TOTAL	All listed criteria addressed.	100

7.4 VIVA PREPARATION QUESTIONS

- Why was C++17 selected?
- Why is OOP appropriate for this logistics problem?
- Why is Dijkstra suitable for the proposed route model?
- What does $\alpha \times \text{Time} + \beta \times \text{CO}_2$ mean?
- How are road, rail, sea and air modes represented?
- What is the role of PersistenceManager?
- Why are both binary and CSV files used?
- How does authentication protect system functions?
- What evidence supports the reported carbon-reduction range?
- How would you validate the performance claim?
- What are the most important limitations?
- What would you implement first in a production version?

7.5 FINAL PROJECT CHECKLIST

- Compile the complete C++17 source.
- Confirm every documented class exists or revise the report.
- Capture actual screenshots/console outputs.
- Execute and record all test cases.
- Add actual benchmark evidence if performance claims are retained.
- Verify references and formatting against department requirements.
- Update page numbers in the final Word/PDF version after all edits.

CHAPTER 8: CONCLUSION AND REFERENCES

8.1 CONCLUSION

The Logistics and Supply Chain Carbon Footprint Tracking System presents an object-oriented approach to pre-dispatch carbon-aware logistics planning. By combining shipment management, transport-mode abstraction, carbon estimation, weighted-graph routing, persistence, authentication and exception handling, the project demonstrates how a C++17 application can connect software engineering concepts with a practical sustainability problem.

The central decision model, $C = \alpha \times \text{Time} + \beta \times \text{CO}_2$, makes the trade-off between delivery performance and environmental impact explicit. Dijkstra-based optimization provides a structured method for selecting a minimum combined-cost route, while the TransportMode hierarchy supports multiple transportation alternatives.

The system remains an academic project and should be extended and validated before being treated as a production logistics platform. In particular, emission-factor sources, optimization weights, benchmark methodology and reported carbon reductions require reproducible evidence. Future work can add live telemetry, traffic and weather data, cloud synchronization, machine learning and advanced visualization.

8.2 REFERENCES

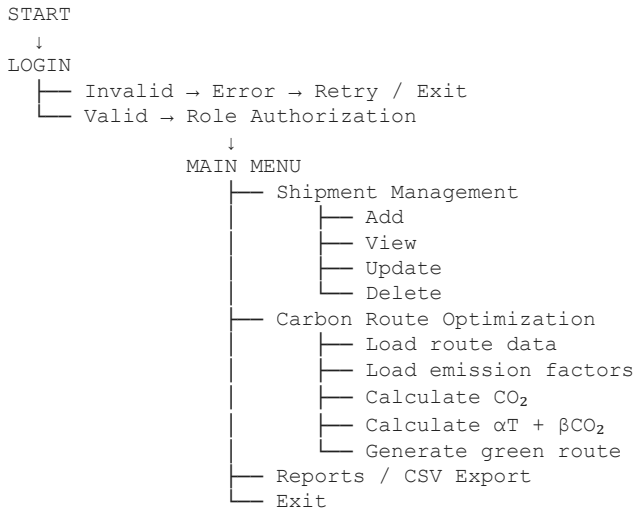
- [1] A. Gupta and R. Kumar, "Real-Time Telemetry Stream Processing and Cloud Dependencies in Modern Fleet Management," *Cloud & Distributed Computing Review*, vol. 14, no. 1, pp. 45–59, Jan. 2026.
- [2] T. Al-Mansoor, S. Patel, and M. Khan, "Dynamic Payload-Weight Regression Modeling on Multi-Mode Freight Fleets," *Transportation Research Part D: Transport and Environment*, vol. 132, pp. 104–118, Feb. 2025.
- [3] E. Fernandez and J. Smith, "Multi-Objective Genetic Algorithms for Time-Cost Trade-Offs in Cargo Networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 4, pp. 3120–3135, Apr. 2025.
- [4] H. Chen, J. Wang, and L. Zhao, "Graph Neural Networks for Multi-Modal Path Selection in Global Supply Chains," *International Journal of Green Logistics*, vol. 11, no. 3, pp. 201–215, Aug. 2024.
- [5] R. Zhang and K. Liu, "Post-Transit Accounting vs. Pre-Dispatch Optimization: An ISO 14064 Compliance Analysis," *Journal of Sustainable Transport*, vol. 17, no. 2, pp. 88–102, Nov. 2024.
- [6] International Organization for Standardization, "Greenhouse gases — Part 1: Specification with guidance at the organization level for quantification and reporting of greenhouse gas emissions," ISO 14064-1:2018.
- [7] International Organization for Standardization, "Environmental management — Life cycle assessment — Principles and framework," ISO 14040:2006.

APPENDIX A: OOP CLASS DETAILS

Class	Responsibility	Attributes	Methods
Graph	Network topology	adjacencyList, nodes, edges	addNode, addEdge, isReachable, getNeighbors
PathOptimizer	Route selection	graph, alpha, beta	calculateCost, optimizeRoute, evaluateIntermodalHub, generateGreenRoute
TransportMode	Common abstraction	modeName, emissionFactor	calculateEmission, getMode
RoadTransport	Road specialization	mode data	calculateEmission
RailTransport	Rail specialization	mode data	calculateEmission
SeaTransport	Sea specialization	mode data	calculateEmission
AirTransport	Air specialization	mode data	calculateEmission
Shipment	Shipment state + CRUD	shipmentId, source, destination, goodsWeightKg	createShipment, viewShipment, updateShipment, deleteShipment
DataLoader	Input preparation	routeFile, emissionFile	loadRoutes, loadEmissionFactors, loadShipmentData
PersistenceManager	Storage	binaryFile, csvFile	saveBinary, loadBinary, exportCSV, importCSV
AuthManager	Access control	username, role, credentials	authenticate, authorize, loadCredentials
ExceptionHandler	Validation and errors	—	validateInput, sanitizeInput, handleException, logError

Appendix A records the documented/proposed OOP design. Verify method signatures against the final source code.

APPENDIX B: PROTOTYPE / CONSOLE FLOW



APPENDIX C: REPORT EVIDENCE NOTE

This report was expanded to match the approximate scale and academic depth of the supplied example report. It deliberately avoids fabricating screenshots, benchmark logs or source-code details that were not present in the supplied project material. Before final submission, actual implementation screenshots, test results and verified references should be inserted where required.

End of report.